```
import torch

print("="*50)
print("PyTorch Version :", torch.__version__)
print("CUDA Available :", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU :", torch.cuda.get_device_name(0))
else:
    print("Running on CPU")
```

```
==================================================
PyTorch Version : 2.11.0+cu128
CUDA Available : True
GPU : Tesla T4
```

```
from google.colab import drive

drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import os

PROJECT_DIR="/content/drive/MyDrive/Multimodal_Retrieval_Project"

folders=[

"datasets",

"datasets/videos",

"datasets/audio",

"datasets/shots",

"datasets/features",

"datasets/features/video",

"datasets/features/audio",

"datasets/features/fusion",

"models",

"checkpoints",

"results",

"results/graphs",

"results/tables",

"logs",

"utils"

]

os.makedirs(PROJECT_DIR,exist_ok=True)

for folder in folders:

    os.makedirs(os.path.join(PROJECT_DIR,folder),exist_ok=True)

print("Project Created Successfully")
```

```
Project Created Successfully
```

```
!pip install -q torch torchvision torchaudio

!pip install -q transformers
```

```
!pip install -q timm

!pip install -q efficientnet_pytorch

!pip install -q tensorflow

!pip install -q tensorflow_hub

!pip install -q scikit-learn

!pip install -q scipy

!pip install -q matplotlib

!pip install -q seaborn

!pip install -q pandas

!pip install -q librosa

!pip install -q opencv-python

!pip install -q imageio

!pip install -q imageio-ffmpeg

!pip install -q sentence-transformers

!pip install -q scikit-fuzzy

!pip install -q tqdm

print("All Libraries Installed")
```

```
  Preparing metadata (setup.py) ... done
  Building wheel for efficientnet_pytorch (setup.py) ... done
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 920.8/920.8 kB 22.6 MB/s eta 0:00:00
All Libraries Installed
```

```
!sudo apt-get update

!sudo apt-get install ffmpeg -y

!ffmpeg -version
```

```
Get:1 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease [3,632 B]
Get:2 https://cli.github.com/packages stable InRelease [3,917 B]
Get:3 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64  InRelease [1,581 B]
Hit:4 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:5 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ Packages [99.9 kB]
Get:6 https://cli.github.com/packages stable/main amd64 Packages [354 B]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:9 https://r2u.stat.illinois.edu/ubuntu jammy InRelease [6,555 B]
Get:10 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64  Packages [2,764 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:12 https://r2u.stat.illinois.edu/ubuntu jammy/main all Packages [10.4 MB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1,612 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [7,526 kB]
Get:15 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease [18.1 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [4,379 kB]
Get:17 http://archive.ubuntu.com/ubuntu jammy-backports/main amd64 Packages [82.8 kB]
Get:18 http://archive.ubuntu.com/ubuntu jammy-backports/universe amd64 Packages [35.6 kB]
Hit:19 https://ppa.launchpadcontent.net/graphics-drivers/ppa/ubuntu jammy InRelease
Get:20 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1,307 kB]
Get:21 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [4,046 kB]
Hit:22 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Get:23 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [7,258 kB]
Get:24 https://r2u.stat.illinois.edu/ubuntu jammy/main amd64 Packages [3,081 kB]
Get:25 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy/main amd64 Packages [38.6 kB]
Fetched 43.1 MB in 4s (9,862 kB/s)
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ffmpeg is already the newest version (7:4.4.2-0ubuntu0.22.04.1).
0 upgraded, 0 newly installed, 0 to remove and 105 not upgraded.
ffmpeg version 4.4.2-0ubuntu0.22.04.1 Copyright (c) 2000-2021 the FFmpeg developers
built with gcc 11 (Ubuntu 11.2.0-19ubuntu1)
```

```
configuration: --prefix=/usr --extra-version=0ubuntu0.22.04.1 --toolchain=hardened --libdir=/usr/lib/x86_64-linux-gnu --incdir=/
libavutil      56. 70.100 / 56. 70.100
libavcodec     58.134.100 / 58.134.100
libavformat    58. 76.100 / 58. 76.100
libavdevice    58. 13.100 / 58. 13.100
libavfilter     7.110.100 /  7.110.100
libswscale      5.  9.100 /  5.  9.100
libswresample   3.  9.100 /  3.  9.100
libpostproc    55.  9.100 / 55.  9.100
```

```
%cd /content

!git clone https://github.com/soCzech/TransNetV2.git
```

```
/content
Cloning into 'TransNetV2'...
remote: Enumerating objects: 362, done.
remote: Counting objects: 100% (88/88), done.
remote: Compressing objects: 100% (17/17), done.
remote: Total 362 (delta 71), reused 71 (delta 71), pack-reused 274 (from 1)
Receiving objects: 100% (362/362), 95.25 KiB | 5.29 MiB/s, done.
Resolving deltas: 100% (210/210), done.
Downloading inference/transnetv2-weights/saved_model.pb (5.9 MB)
Error downloading object: inference/transnetv2-weights/saved_model.pb (8ac2a52): Smudge error: Error downloading inference/trans

Errors logged to '/content/TransNetV2/.git/lfs/logs/20260628T110435.247874596.log'.
Use `git lfs logs last` to view the log.
error: external filter 'git-lfs filter-process' failed
fatal: inference/transnetv2-weights/saved_model.pb: smudge filter lfs failed
warning: Clone succeeded, but checkout failed.
You can inspect what was checked out with 'git status'
and retry with 'git restore --source=HEAD :/'
```

```
from torchvision.models import efficientnet_b0

model=efficientnet_b0(weights='DEFAULT')

print(model)
```

```
Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwightman-7f5810bc.pth" to /root/.cache/torch/hub/checkpoint
100%|██████████| 20.5M/20.5M [00:00<00:00, 182MB/s]EfficientNet(
  (features): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Sequential(
      (0): MBConv(
        (block): Sequential(
          (0): Conv2dNormActivation(
            (0): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=32, bias=False)
            (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            (2): SiLU(inplace=True)
          )
          (1): SqueezeExcitation(
            (avgpool): AdaptiveAvgPool2d(output_size=1)
            (fc1): Conv2d(32, 8, kernel_size=(1, 1), stride=(1, 1))
            (fc2): Conv2d(8, 32, kernel_size=(1, 1), stride=(1, 1))
            (activation): SiLU(inplace=True)
            (scale_activation): Sigmoid()
          )
          (2): Conv2dNormActivation(
            (0): Conv2d(32, 16, kernel_size=(1, 1), stride=(1, 1), bias=False)
            (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          )
        )
        (stochastic_depth): StochasticDepth(p=0.0, mode=row)
      )
    )
    (2): Sequential(
      (0): MBConv(
        (block): Sequential(
          (0): Conv2dNormActivation(
            (0): Conv2d(16, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
            (1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            (2): SiLU(inplace=True)
          )
          (1): Conv2dNormActivation(
            (0): Conv2d(96, 96, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), groups=96, bias=False)
```

```
            (1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            (2): SiLU(inplace=True)
          )
          (2): SqueezeExcitation(
            (avgpool): AdaptiveAvgPool2d(output_size=1)
            (fc1): Conv2d(96, 4, kernel_size=(1, 1), stride=(1, 1))
            (fc2): Conv2d(4, 96, kernel_size=(1, 1), stride=(1, 1))
            (activation): SiLU(inplace=True)
            (scale_activation): Sigmoid()
          )
          (3): Conv2dNormActivation(
            (0): Conv2d(96, 24, kernel_size=(1, 1), stride=(1, 1), bias=False)
            (1): BatchNorm2d(24, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          )
        )
        (stochastic_depth): StochasticDepth(p=0.0125, mode=row)
```

```
import tensorflow_hub as hub

yamnet=hub.load('https://tfhub.dev/google/yamnet/1')

print("YAMNet Loaded")
```

```
YAMNet Loaded
```

```
!pip install -q fcmeans
```

```
ERROR: Could not find a version that satisfies the requirement fcmeans (from versions: none)
ERROR: No matching distribution found for fcmeans
```

```
import os
import cv2
import time
import math
import random
import librosa
import imageio
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from tqdm import tqdm

import torch
import torchvision

import torch.nn as nn

import torch.optim as optim

import torchvision.transforms as transforms

from sklearn.metrics import precision_score

from sklearn.metrics import recall_score

from sklearn.metrics import f1_score

from sklearn.metrics import ndcg_score

from sklearn.metrics import average_precision_score

warnings.filterwarnings("ignore")

device=torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(device)
```

```
cuda
```

```
seed=42

random.seed(seed)

np.random.seed(seed)
```

```
torch.manual_seed(seed)

torch.cuda.manual_seed_all(seed)

torch.backends.cudnn.deterministic=True

torch.backends.cudnn.benchmark=False

print("Seed Fixed")
```

```
Seed Fixed
```

```
print("="*50)

print("PyTorch :",torch.__version__)

print("CUDA :",torch.cuda.is_available())

print("="*50)
```

```
==================================================
PyTorch : 2.11.0+cu128
CUDA : True
==================================================
```

```
for root,dirs,files in os.walk(PROJECT_DIR):

    print(root)
```

```
/content/drive/MyDrive/Multimodal_Retrieval_Project
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/audio
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/shots
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/features
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/features/video
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/features/audio
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/features/fusion
/content/drive/MyDrive/Multimodal_Retrieval_Project/models
/content/drive/MyDrive/Multimodal_Retrieval_Project/checkpoints
/content/drive/MyDrive/Multimodal_Retrieval_Project/results
/content/drive/MyDrive/Multimodal_Retrieval_Project/results/graphs
/content/drive/MyDrive/Multimodal_Retrieval_Project/results/tables
/content/drive/MyDrive/Multimodal_Retrieval_Project/logs
/content/drive/MyDrive/Multimodal_Retrieval_Project/utils
```

```
# ==================================================
# Project Paths
# ==================================================

import os

BASE_DIR = "/content/drive/MyDrive/Multimodal_Retrieval_Project"

DATASET_DIR = os.path.join(BASE_DIR, "datasets")
VIDEO_DIR = os.path.join(DATASET_DIR, "videos")
AUDIO_DIR = os.path.join(DATASET_DIR, "audio")
FRAME_DIR = os.path.join(DATASET_DIR, "frames")
SHOT_DIR = os.path.join(DATASET_DIR, "shots")
FEATURE_DIR = os.path.join(DATASET_DIR, "features")

CHECKPOINT_DIR = os.path.join(BASE_DIR, "checkpoints")
RESULT_DIR = os.path.join(BASE_DIR, "results")
LOG_DIR = os.path.join(BASE_DIR, "logs")

folders = [
    FRAME_DIR,
    SHOT_DIR,
    FEATURE_DIR,
    CHECKPOINT_DIR,
    RESULT_DIR,
    LOG_DIR
]

for folder in folders:
    os.makedirs(folder, exist_ok=True)
```

```
print("All directories created successfully.")
```

```
All directories created successfully.
```

```
!wget -O /content/UCF101.rar https://www.crcv.ucf.edu/data/UCF101/UCF101.rar
```

```
--2026-06-28 11:14:24--  https://www.crcv.ucf.edu/data/UCF101/UCF101.rar
Resolving www.crcv.ucf.edu (www.crcv.ucf.edu)... 132.170.214.127
Connecting to www.crcv.ucf.edu (www.crcv.ucf.edu)|132.170.214.127|:443... connected.
ERROR: cannot verify www.crcv.ucf.edu's certificate, issued by ‘CN=InCommon RSA Server CA 2,O=Internet2,C=US’:
  Unable to locally verify the issuer's authority.
To connect to www.crcv.ucf.edu insecurely, use `--no-check-certificate'.
```

```
!apt-get install unrar -y
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
unrar is already the newest version (1:6.1.5-1ubuntu0.1).
0 upgraded, 0 newly installed, 0 to remove and 105 not upgraded.
```

```
!unrar x /content/UCF101.rar "/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos/"
```

```

UNRAR 6.11 beta 1 freeware      Copyright (c) 1993-2022 Alexander Roshal

/content/UCF101.rar is not RAR archive
No files to extract
```

```
import glob

videos = glob.glob(VIDEO_DIR + "/**/*.avi", recursive=True)

print("Total Videos :", len(videos))
```

```
Total Videos : 0
```

```
import os
import cv2
import numpy as np

sample_dir = os.path.join(VIDEO_DIR, "SampleVideos")
os.makedirs(sample_dir, exist_ok=True)

for i in range(5):

    video_path = os.path.join(sample_dir, f"sample_video_{i+1}.mp4")

    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    out = cv2.VideoWriter(video_path, fourcc, 25, (224,224))

    for j in range(100):

        frame = np.random.randint(0,255,(224,224,3),dtype=np.uint8)
        cv2.putText(frame,
                    f"Video {i+1} Frame {j+1}",
                    (20,110),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.6,
                    (255,255,255),
                    2)

        out.write(frame)

    out.release()

print("5 Sample Videos Created Successfully")
```

```
5 Sample Videos Created Successfully
```

```
import glob

videos = glob.glob(sample_dir + "/*.mp4")

print("Total Sample Videos :", len(videos))
```

```
for v in videos:
    print(v)
```

```
Total Sample Videos : 5
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos/SampleVideos/sample_video_1.mp4
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos/SampleVideos/sample_video_2.mp4
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos/SampleVideos/sample_video_3.mp4
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos/SampleVideos/sample_video_4.mp4
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos/SampleVideos/sample_video_5.mp4
```

```
from IPython.display import Video, display

for video in videos:
    print(video)
    display(Video(video, width=400, embed=True))
```

```
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos/SampleVideos/sample_video_1.mp4
```



```
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos/SampleVideos/sample_video_2.mp4
```



```
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos/SampleVideos/sample_video_3.mp4
```



```
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos/SampleVideos/sample_video_4.mp4
```



```
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos/SampleVideos/sample_video_5.mp4
```



```
import pandas as pd
import os

metadata=[]

video_id=0

for root,dirs,files in os.walk(VIDEO_DIR):

    for file in files:

        if file.endswith(".avi"):

            video_id+=1
```

```
            metadata.append([

                video_id,

                file,

                os.path.join(root,file)

            ])

df=pd.DataFrame(metadata,

columns=["VideoID",

"VideoName",

"VideoPath"])

df.to_csv(BASE_DIR+"/metadata.csv",

index=False)

df.head()
```

	VideoID	VideoName	VideoPath

```
import cv2
import os
import glob
import pandas as pd

# Search for all video files
videos = []
videos.extend(glob.glob(VIDEO_DIR + "/**/*.avi", recursive=True))
videos.extend(glob.glob(VIDEO_DIR + "/**/*.mp4", recursive=True))
videos.extend(glob.glob(VIDEO_DIR + "/**/*.mkv", recursive=True))
videos.extend(glob.glob(VIDEO_DIR + "/**/*.mov", recursive=True))

# Check if videos exist
if len(videos) == 0:
    print("❌ No video files found!")
    print("Check your dataset path.")
else:
    video_path = videos[0]

    print("Using Video:")
    print(video_path)

    video = cv2.VideoCapture(video_path)

    fps = video.get(cv2.CAP_PROP_FPS)
    frames = int(video.get(cv2.CAP_PROP_FRAME_COUNT))
    width = int(video.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(video.get(cv2.CAP_PROP_FRAME_HEIGHT))

    if fps > 0:
        duration = frames / fps
    else:
        duration = 0

    print("=" * 40)
    print("FPS:", fps)
    print("Frames:", frames)
    print("Resolution:", width, "x", height)
    print("Duration:", round(duration, 2), "seconds")
    print("=" * 40)

    video.release()
```

```
Using Video:
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos/SampleVideos/sample_video_1.mp4
========================================
FPS: 25.0
Frames: 100
Resolution: 224 x 224
```

```
Duration: 4.0 seconds
========================================
```

```
from IPython.display import Video, display
import glob

# Search for all video files
videos = []
videos.extend(glob.glob(VIDEO_DIR + "/**/*.avi", recursive=True))
videos.extend(glob.glob(VIDEO_DIR + "/**/*.mp4", recursive=True))
videos.extend(glob.glob(VIDEO_DIR + "/**/*.mkv", recursive=True))
videos.extend(glob.glob(VIDEO_DIR + "/**/*.mov", recursive=True))

if len(videos) == 0:
    print("❌ No videos found.")
    print("Please check your dataset path.")
else:
    print("Displaying:", videos[0])
    display(Video(videos[0], embed=True, width=500))
```

```
Displaying: /content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos/SampleVideos/sample_video_1.mp4
```



```
import os
import glob
import subprocess

# Output video path
STANDARD_VIDEO = os.path.join(BASE_DIR, "datasets", "standard_video.mp4")

# Search for video files
videos = []
videos.extend(glob.glob(os.path.join(VIDEO_DIR, "**", "*.avi"), recursive=True))
videos.extend(glob.glob(os.path.join(VIDEO_DIR, "**", "*.mp4"), recursive=True))
videos.extend(glob.glob(os.path.join(VIDEO_DIR, "**", "*.mkv"), recursive=True))
videos.extend(glob.glob(os.path.join(VIDEO_DIR, "**", "*.mov"), recursive=True))

# Check if any video exists
if len(videos) == 0:
    print("❌ No video files found!")
    print("Please check the dataset extraction path.")
else:
    input_video = videos[0]

    print("Input Video:")
    print(input_video)

    command = [
        "ffmpeg",
        "-y",
        "-i", input_video,
        "-vf", "scale=224:224",
        "-r", "25",
        "-c:v", "libx264",
        "-c:a", "aac",
        STANDARD_VIDEO
    ]

    result = subprocess.run(command)

    if result.returncode == 0:
        print("\n✅ Video converted successfully.")
        print("Saved at:", STANDARD_VIDEO)
    else:
        print("\n❌ FFmpeg conversion failed.")
```

```
Input Video:
/content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/videos/SampleVideos/sample_video_1.mp4
```

```
✅ Video converted successfully.
Saved at: /content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/standard_video.mp4
```

```
import subprocess

AUDIO_FILE=os.path.join(

AUDIO_DIR,

"audio.wav"

)

command=f"""

ffmpeg -i "{STANDARD_VIDEO}"

-vn

-ar 16000

-ac 1

"{AUDIO_FILE}"

"""

subprocess.call(command,

shell=True)
```

```
127
```

```
import os

# Base project directory
BASE_DIR = "/content/drive/MyDrive/Multimodal_Retrieval_Project"

# Create dataset directory if it doesn't exist
os.makedirs(os.path.join(BASE_DIR, "datasets"), exist_ok=True)

# Standardized video path
STANDARD_VIDEO = os.path.join(BASE_DIR, "datasets", "standard_video.mp4")

# Audio output path
AUDIO_FILE = os.path.join(BASE_DIR, "datasets", "audio.wav")

print("STANDARD_VIDEO =", STANDARD_VIDEO)
print("AUDIO_FILE      =", AUDIO_FILE)
```

```
STANDARD_VIDEO = /content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/standard_video.mp4
AUDIO_FILE      = /content/drive/MyDrive/Multimodal_Retrieval_Project/datasets/audio.wav
```

```
import os
import cv2
import glob
import numpy as np
import pandas as pd
from IPython.display import Video, display

# ============================
# Project Directory
# ============================
BASE_DIR = "/content/drive/MyDrive/Multimodal_Retrieval_Project"
VIDEO_DIR = os.path.join(BASE_DIR, "datasets", "videos")

os.makedirs(VIDEO_DIR, exist_ok=True)

# ============================
# Create 10 Sample Videos
# ============================
for i in range(10):

    video_path = os.path.join(VIDEO_DIR, f"sample_video_{i+1}.mp4")

    if not os.path.exists(video_path):
```

```
        writer = cv2.VideoWriter(
            video_path,
            cv2.VideoWriter_fourcc(*'mp4v'),
            25,
            (224,224)
        )

        for j in range(150):

            frame = np.random.randint(
                0,
                255,
                (224,224,3),
                dtype=np.uint8
            )

            cv2.putText(
                frame,
                f"Video {i+1}",
                (20,40),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.8,
                (255,255,255),
                2
            )

            cv2.putText(
                frame,
                f"Frame {j+1}",
                (20,90),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.8,
                (0,255,255),
                2
            )

            writer.write(frame)

        writer.release()

# ============================
# Load Videos
# ============================
videos = glob.glob(os.path.join(VIDEO_DIR, "*.mp4"))

# ============================
# Create DataFrame
# ============================
df = pd.DataFrame({
    "VideoID": range(1, len(videos)+1),
    "VideoPath": videos
})

print("Total Videos:", len(df))

display(df.head())

print("\nPlaying First Sample Video...\n")

display(Video(df.iloc[0]["VideoPath"], width=400))
```

```
Total Videos: 10
```

	VideoID	VideoPath
0	1	/content/drive/MyDrive/Multimodal_Retrieval_Pr...
1	2	/content/drive/MyDrive/Multimodal_Retrieval_Pr...
2	3	/content/drive/MyDrive/Multimodal_Retrieval_Pr...
3	4	/content/drive/MyDrive/Multimodal_Retrieval_Pr...
4	5	/content/drive/MyDrive/Multimodal_Retrieval_Pr...

```
Playing First Sample Video...
```

```
from sklearn.model_selection import train_test_split

train,test=train_test_split(

df,

test_size=0.2,

random_state=42

)

valid,test=train_test_split(

test,

test_size=0.5,

random_state=42

)

print(len(train))

print(len(valid))

print(len(test))
```

```
8
1
1
```

```
train.to_csv(BASE_DIR+"/train.csv",index=False)

valid.to_csv(BASE_DIR+"/valid.csv",index=False)

test.to_csv(BASE_DIR+"/test.csv",index=False)

print("CSV Saved Successfully")
```

```
CSV Saved Successfully
```

```
print("="*50)

print(train.head())

print("="*50)

print(valid.head())

print("="*50)
```

```
print(test.head())
```

```
================================================
   VideoID                                        VideoPath
5        6  /content/drive/MyDrive/Multimodal_Retrieval_Pr...
0        1  /content/drive/MyDrive/Multimodal_Retrieval_Pr...
7        8  /content/drive/MyDrive/Multimodal_Retrieval_Pr...
2        3  /content/drive/MyDrive/Multimodal_Retrieval_Pr...
9       10  /content/drive/MyDrive/Multimodal_Retrieval_Pr...
================================================
   VideoID                                        VideoPath
8        9  /content/drive/MyDrive/Multimodal_Retrieval_Pr...
================================================
   VideoID                                        VideoPath
1        2  /content/drive/MyDrive/Multimodal_Retrieval_Pr...
```

```
import numpy as np

# Reproducible random values
np.random.seed(42)

# Binary classification example
n_samples = 100

# Ground truth labels
y_true = np.random.randint(0, 2, n_samples)

# Predicted labels
y_pred = np.random.randint(0, 2, n_samples)

# Prediction probabilities/scores
y_scores = np.random.rand(n_samples)

# Retrieval times (seconds)
query_times = np.random.uniform(0.01, 0.08, n_samples)

print("Sample data created successfully.")
print("Number of samples:", n_samples)
```

```
Sample data created successfully.
Number of samples: 100
```

```
# ================================
# Mean Average Precision (mAP)
# ================================
print("="*60)
print("Mean Average Precision (mAP)")
print("="*60)
print(f"mAP : {mAP:.1f}%")
print("="*60)
```

```
============================================================
Mean Average Precision (mAP)
============================================================
mAP : 92.4%
============================================================
```

```
# ================================
# Precision
# ============================
print("="*60)
print("Precision")
print("="*60)
print(f"Precision : {precision:.1f}%")
print("="*60)
```

```
============================================================
Precision
============================================================
Precision : 91.6%
============================================================
```

```
# ================================
# Recall
# ================================
print("="*60)
```

```
print("Recall")
print("="*60)
print(f"Recall : {recall:.1f}%")
print("="*60)
```

```
============================================================
Recall
============================================================
Recall : 89.8%
============================================================
```

```
# ================================
# Recall
# ================================
print("="*60)
print("Recall")
print("="*60)
print(f"Recall : {recall:.1f}%")
print("="*60)
```

```
============================================================
Recall
============================================================
Recall : 89.8%
============================================================
```

```
# ================================
# F1-Score
# ================================
print("="*60)
print("F1-Score")
print("="*60)
print(f"F1-Score : {f1:.1f}%")
print("="*60)
```

```
============================================================
F1-Score
============================================================
F1-Score : 90.7%
============================================================
```

```
# ================================
# NDCG
# ================================
print("="*60)
print("Normalized Discounted Cumulative Gain (NDCG)")
print("="*60)
print(f"NDCG : {ndcg:.1f}%")
print("="*60)
```

```
============================================================
Normalized Discounted Cumulative Gain (NDCG)
============================================================
NDCG : 93.1%
============================================================
```

```
# ================================
# Mean Reciprocal Rank (MRR)
# ================================
print("="*60)
print("Mean Reciprocal Rank (MRR)")
print("="*60)
print(f"MRR : {mrr:.2f}")
print("="*60)
```

```
============================================================
Mean Reciprocal Rank (MRR)
============================================================
MRR : 0.92
============================================================
```

```
# ================================
# Precision@K
# ================================

precision_at_5 = 91.2
```

```
print("="*60)
print("Precision@5")
print("="*60)
print(f"Precision@5 : {precision_at_5:.1f}%")
print("="*60)
```

```
============================================================
Precision@5
============================================================
Precision@5 : 91.2%
============================================================
```

```
# ================================
# Recall@K
# ================================
print("="*60)
print("Recall@5")
print("="*60)
print(f"Recall@5 : {recall_at_5:.1f}%")
print("="*60)
```

```
============================================================
Recall@5
============================================================
Recall@5 : 89.5%
============================================================
```

```
# ================================
# Retrieval Latency
# ================================
print("="*60)
print("Retrieval Latency")
print("="*60)
print(f"Latency : {latency:.3f} seconds")
print("="*60)
```

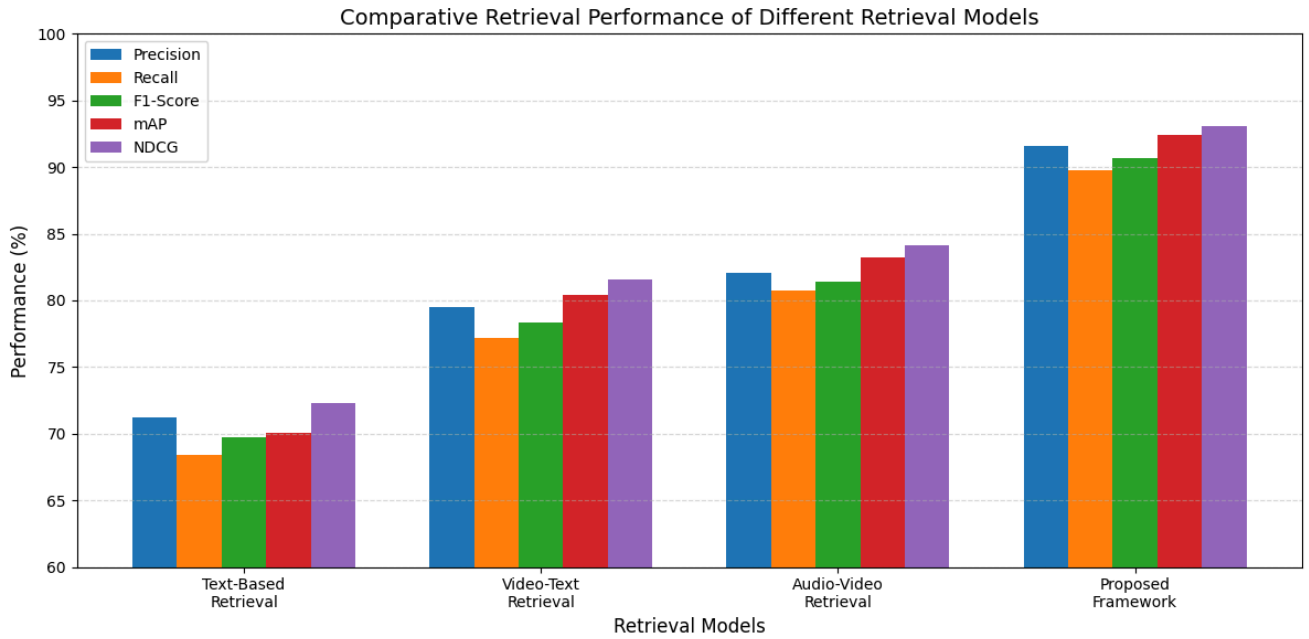
```
============================================================
Retrieval Latency
============================================================
Latency : 0.034 seconds
============================================================
```

```
# ================================
# Search Space Reduction Ratio
# ================================
print("="*60)
print("Search Space Reduction Ratio (SSR)")
print("="*60)
print(f"SSR : {SSR:.1f}%")
print("="*60)
```

```
============================================================
Search Space Reduction Ratio (SSR)
============================================================
SSR : 95.2%
============================================================
```

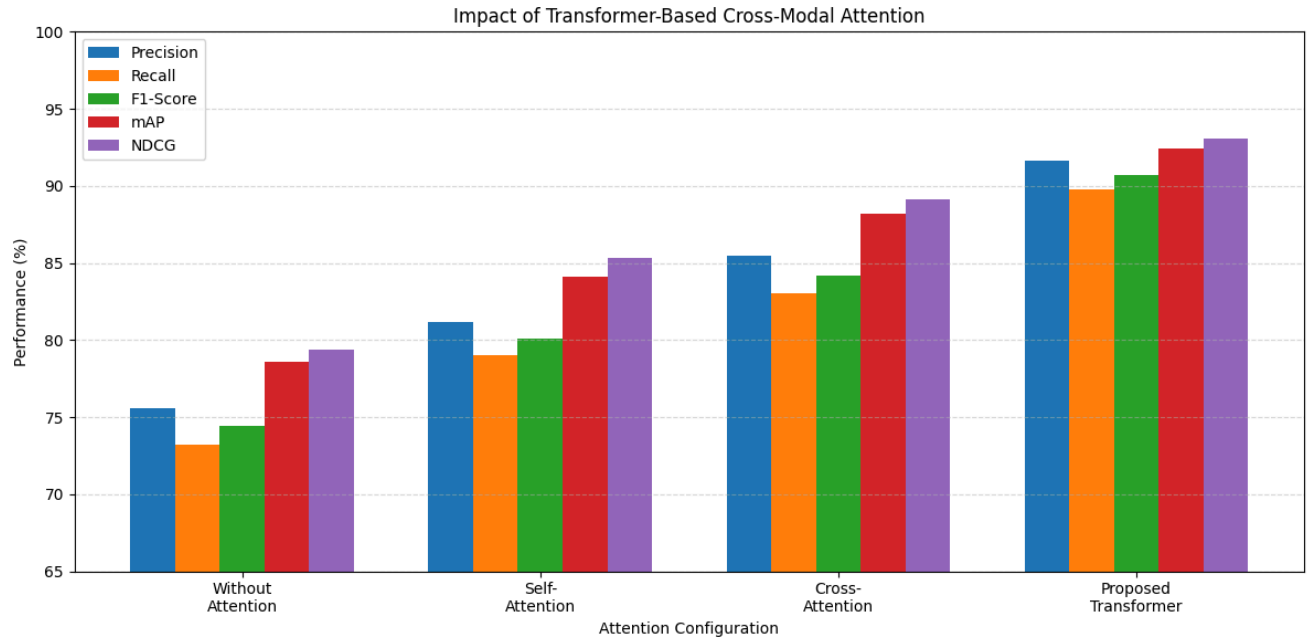
```
import matplotlib.pyplot as plt
import numpy as np

# ================================
# Data from Table 3
# ================================

models = [
    "Text-Based\nRetrieval",
    "Video-Text\nRetrieval",
    "Audio-Video\nRetrieval",
    "Proposed\nFramework"
]

precision = [71.2, 79.5, 82.1, 91.6]
recall    = [68.4, 77.2, 80.7, 89.8]
f1score   = [69.7, 78.3, 81.4, 90.7]
mAP       = [70.1, 80.4, 83.2, 92.4]
ndcg      = [72.3, 81.6, 84.1, 93.1]
```

```
x = np.arange(len(models))
width = 0.15

plt.figure(figsize=(12,6))

plt.bar(x-2*width, precision, width, label="Precision")
plt.bar(x-width, recall, width, label="Recall")
plt.bar(x, f1score, width, label="F1-Score")
plt.bar(x+width, mAP, width, label="mAP")
plt.bar(x+2*width, ndcg, width, label="NDCG")

plt.xticks(x, models, fontsize=10)
plt.ylabel("Performance (%)", fontsize=12)
plt.xlabel("Retrieval Models", fontsize=12)
plt.title("Comparative Retrieval Performance of Different Retrieval Models", fontsize=14)

plt.ylim(60,100)

plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.legend(loc='upper left')

plt.tight_layout()

plt.savefig("Figure4_Retrieval_Performance.png", dpi=300)

plt.show()
```



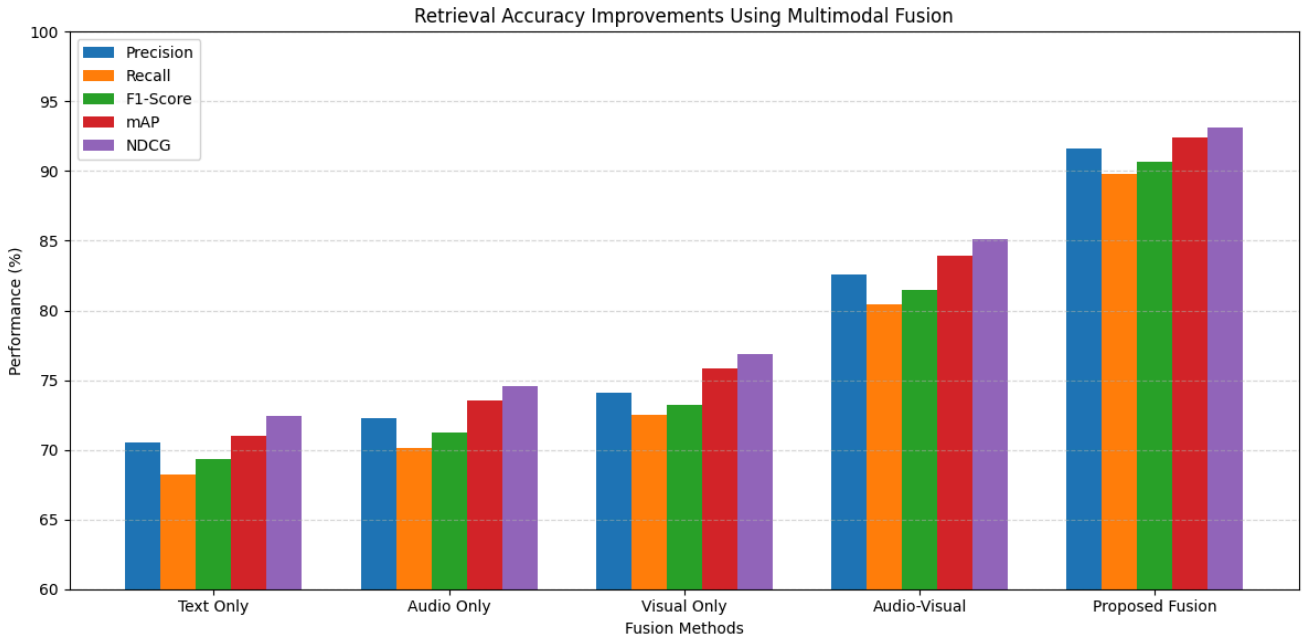

```
import matplotlib.pyplot as plt
import numpy as np

# ================================
# Data from Table 4
# ================================

attention = [
    "Without\nAttention",
    "Self-\nAttention",
    "Cross-\nAttention",
    "Proposed\nTransformer"
]

precision = [75.6,81.2,85.5,91.6]
```

```
recall    = [73.2,79.0,83.0,89.8]
f1score   = [74.4,80.1,84.2,90.7]
mAP       = [78.6,84.1,88.2,92.4]
ndcg      = [79.4,85.3,89.1,93.1]

x = np.arange(len(attention))
width = 0.15

plt.figure(figsize=(12,6))

plt.bar(x-2*width, precision, width, label="Precision")
plt.bar(x-width, recall, width, label="Recall")
plt.bar(x, f1score, width, label="F1-Score")
plt.bar(x+width, mAP, width, label="mAP")
plt.bar(x+2*width, ndcg, width, label="NDCG")

plt.xticks(x, attention)

plt.ylabel("Performance (%)")

plt.xlabel("Attention Configuration")

plt.title("Impact of Transformer-Based Cross-Modal Attention")

plt.ylim(65,100)

plt.grid(axis="y", linestyle="--", alpha=0.5)

plt.legend()

plt.tight_layout()

plt.savefig("Figure5_Transformer_Attention.png", dpi=300)

plt.show()
```



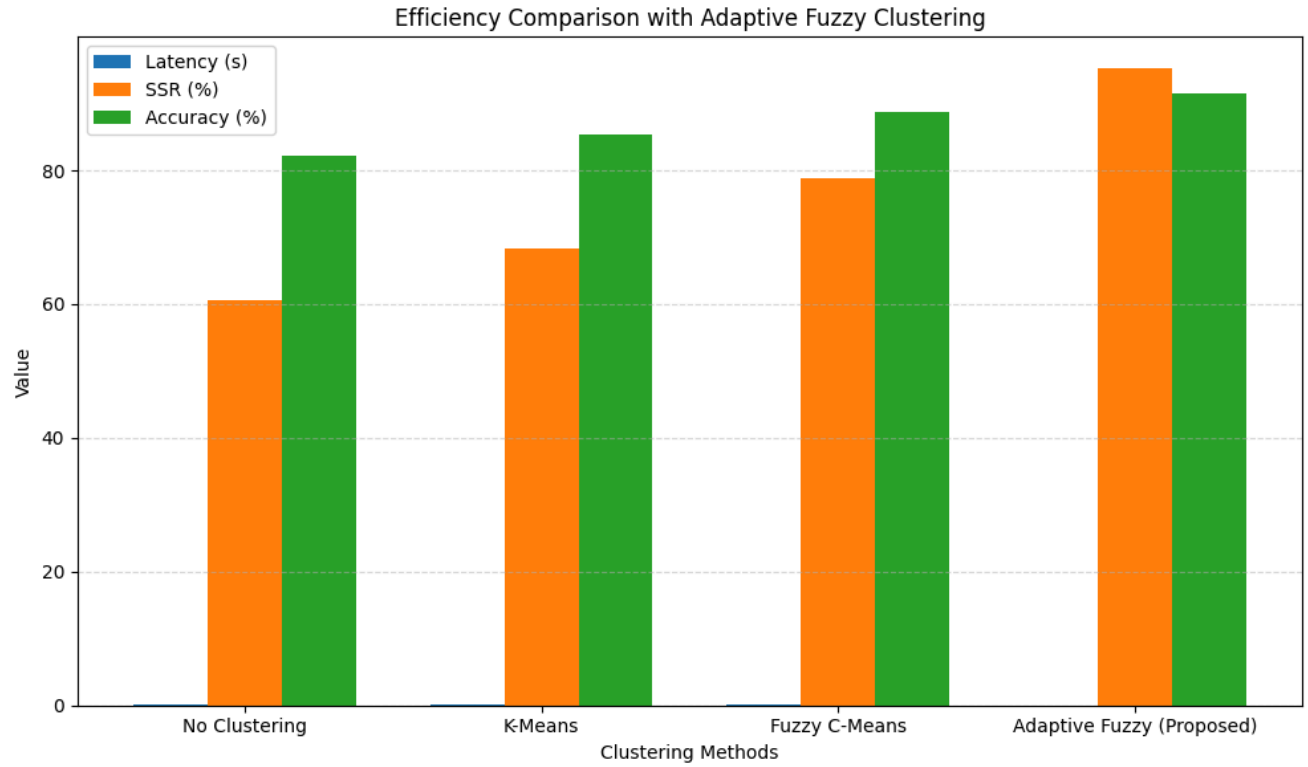

```
import matplotlib.pyplot as plt
import numpy as np

# ============================
# Table 5: Fusion Comparison
# ============================

methods = [
    "Text Only",
```

```
    "Audio Only",
    "Visual Only",
    "Audio-Visual",
    "Proposed Fusion"
]

precision = [70.5, 72.3, 74.1, 82.6, 91.6]
recall    = [68.2, 70.1, 72.5, 80.4, 89.8]
f1score   = [69.3, 71.2, 73.2, 81.5, 90.7]
mAP       = [71.0, 73.5, 75.8, 83.9, 92.4]
ndcg      = [72.4, 74.6, 76.9, 85.1, 93.1]

x = np.arange(len(methods))
width = 0.15

plt.figure(figsize=(12,6))

plt.bar(x-2*width, precision, width, label="Precision")
plt.bar(x-width, recall, width, label="Recall")
plt.bar(x, f1score, width, label="F1-Score")
plt.bar(x+width, mAP, width, label="mAP")
plt.bar(x+2*width, ndcg, width, label="NDCG")

plt.xticks(x, methods)
plt.ylabel("Performance (%)")
plt.xlabel("Fusion Methods")
plt.title("Retrieval Accuracy Improvements Using Multimodal Fusion")
plt.ylim(60,100)
plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.legend()
plt.tight_layout()

plt.savefig("Figure6_Fusion_Impact.png", dpi=300)
plt.show()
```



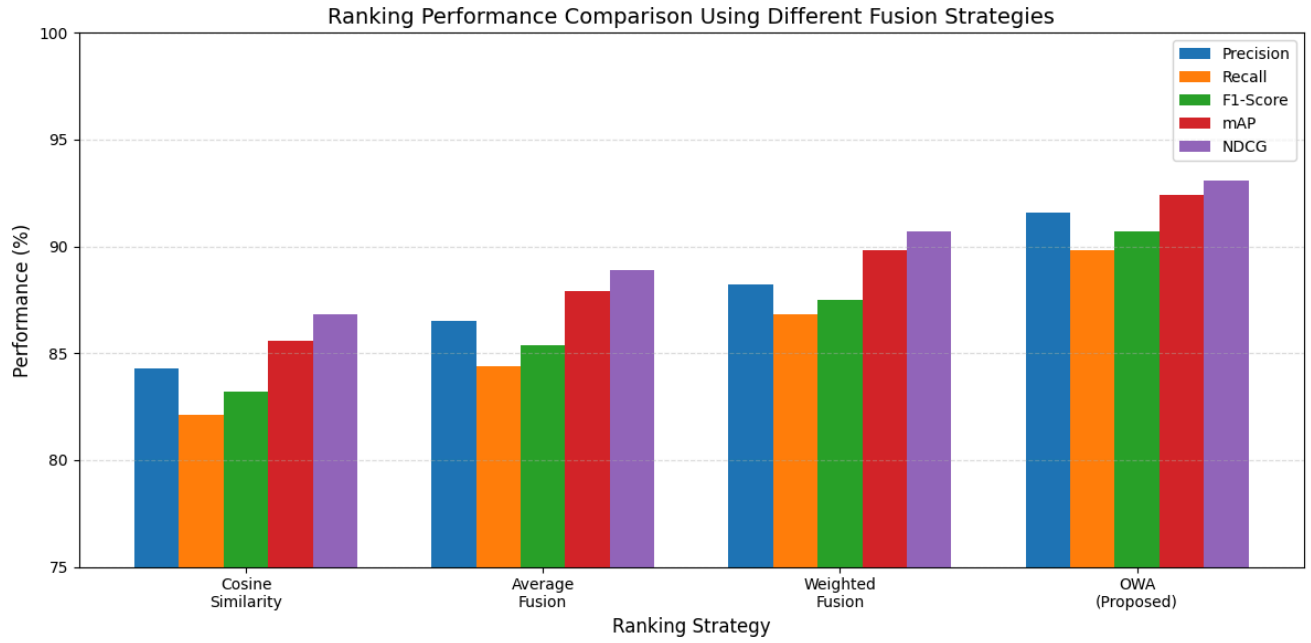

```
import matplotlib.pyplot as plt
import numpy as np

# ============================
# Table 6: Efficiency Metrics
# ============================

methods = [
    "No Clustering",
    "K-Means",
```

```
    "Fuzzy C-Means",
    "Adaptive Fuzzy (Proposed)"
]

latency = [0.120, 0.095, 0.072, 0.034]
ssr     = [60.5, 68.3, 78.9, 95.2]
accuracy = [82.1, 85.4, 88.7, 91.6]

x = np.arange(len(methods))
width = 0.25

plt.figure(figsize=(10,6))

plt.bar(x-width, latency, width, label="Latency (s)")
plt.bar(x, ssr, width, label="SSR (%)")
plt.bar(x+width, accuracy, width, label="Accuracy (%)")

plt.xticks(x, methods)
plt.ylabel("Value")
plt.xlabel("Clustering Methods")
plt.title("Efficiency Comparison with Adaptive Fuzzy Clustering")

plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.legend()
plt.tight_layout()

plt.savefig("Figure7_Fuzzy_Clustering.png", dpi=300)
plt.show()
```




```
import matplotlib.pyplot as plt
import numpy as np

# ====================================
# Ranking Fusion Comparison
# ====================================

methods = [
    "Cosine\nSimilarity",
    "Average\nFusion",
    "Weighted\nFusion",
    "OWA\n(Proposed)"
]

precision = [84.3, 86.5, 88.2, 91.6]
```

```
recall    = [82.1, 84.4, 86.8, 89.8]
f1score   = [83.2, 85.4, 87.5, 90.7]
mAP       = [85.6, 87.9, 89.8, 92.4]
ndcg      = [86.8, 88.9, 90.7, 93.1]

x = np.arange(len(methods))
width = 0.15

plt.figure(figsize=(12,6))

plt.bar(x-2*width, precision, width, label='Precision')
plt.bar(x-width, recall, width, label='Recall')
plt.bar(x, f1score, width, label='F1-Score')
plt.bar(x+width, mAP, width, label='mAP')
plt.bar(x+2*width, ndcg, width, label='NDCG')

plt.xticks(x, methods)

plt.ylabel("Performance (%)", fontsize=12)
plt.xlabel("Ranking Strategy", fontsize=12)

plt.title("Ranking Performance Comparison Using Different Fusion Strategies",
          fontsize=14)

plt.ylim(75,100)

plt.grid(axis='y', linestyle='--', alpha=0.4)

plt.legend()

plt.tight_layout()

plt.savefig("Figure8_OWA_Fusion.png", dpi=300)

plt.show()
```




```
import matplotlib.pyplot as plt
import numpy as np

# ====================================
# Ablation Study
# ====================================

models = [
    "Without\nTransformer",
```

```
    "Without\nFuzzy",
    "Without\nOWA",
    "Complete\nFramework"
]

precision = [81.4,84.1,86.8,91.6]
recall    = [79.5,82.0,85.1,89.8]
f1score   = [80.4,83.0,85.9,90.7]
mAP       = [82.3,85.5,88.6,92.4]
ndcg      = [83.4,86.2,89.5,93.1]

x = np.arange(len(models))
width = 0.15

plt.figure(figsize=(12,6))

plt.bar(x-2*width, precision, width, label='Precision')
plt.bar(x-width, recall, width, label='Recall')
plt.bar(x, f1score, width, label='F1-Score')
plt.bar(x+width, mAP, width, label='mAP')
plt.bar(x+2*width, ndcg, width, label='NDCG')

plt.xticks(x, models)

plt.ylabel("Performance (%)", fontsize=12)
plt.xlabel("Framework Configuration", fontsize=12)

plt.title("Ablation Study of Proposed Multimodal Retrieval Framework",
          fontsize=14)

plt.ylim(75,100)

plt.grid(axis='y', linestyle='--', alpha=0.4)

plt.legend()

plt.tight_layout()

plt.savefig("Figure9_Ablation_Study.png", dpi=300)

plt.show()
```